

The ARM STAC fix

why Forge's dependency repair matters

Context: BON in a Box / Spark ARM blocker in data > loadFromStac for the BII browser route

What broke

The BII run failed before ecological analysis started. The browser route was blocked while creating a script-specific conda environment for **data_loadFromStac**.

Root cause

Forge found an ARM/linux-aarch64 package availability problem: **r-gdalcubes=0.7.1** and **r-proj** could not be solved by libmamba.

Permanent fix

The fix moves the route away from impossible per-script conda specs and into the already validated ARM **rbase** path, with source-installed R packages where needed.

What Forge found

This was not caused by the student's BII input, the chosen area, or the browser settings. The run died earlier: the runner tried to build a new conda environment for **data > loadFromStac**, and libmamba could not solve the requested R geospatial packages for **linux-aarch64**.

```
Creating new conda environment data_loadFromStac...
error libmamba Could not solve for environment specs
The following packages are incompatible
- r-gdalcubes =0.7.1 does not exist
- r-proj does not exist
FAILED
Error: output.json file not found
```

1 Browser launches BII workflow

2 data > loadFromStac asks for script env

3 ARM conda cannot provide pinned R geospatial packages

4 Analysis never starts; no output.json



Important nuance: this is the same class of issue as the earlier DuckDB ARM blocker, but not the same package. The shared pattern is fragile per-script conda dependencies on ARM, not a broken biodiversity model.

The fix Forge applied

Forge turned the local workaround into a repo/runtime repair: remove unsolvable per-script conda environments, run the affected routes through the validated ARM R base environment, and make the runner capable of source-installing missing R packages.



loadFromStac.yml: removed the script-specific conda env. The route now uses validated ARM **rbase**, matching the existing GBIFHeatmapFromSTAC route for the same R script.



zonal_stats.yml: moved to **rbase** as well, after end-to-end BII validation surfaced the same ARM conda gap for **r-exactextractr**.



runners/conda/conda-dockerfile: source-installs missing R packages into rbase when needed: **gdalcubes** and **exactextractr**.



A local runner image was recommitted as **ndforge/conda-diagnostic:arm64-r-extra-py** at sha256:a98350e...f98ce, and a student note was added: **SPARK_ARM64_STAC_GDALCUBES_STUDENT_NOTE.md**.

Verification evidence

✓ YAML / route

Both changed script YAML files parse successfully and no longer contain **conda:** blocks.

✓ Image smoke

Docker smoke passed: **gdalcubes**, **exactextractr**, **sf**, **terra**, **rstac**, and **tidyr** load from rbase.

✓ STAC output

The browser/gateway route for **data > loadFromStac** writes **bii_nhm_2020-01-01.tif** and a valid **output.json** without creating **data_loadFromStac**.



Full BII smoke: Zuid-Holland with valid years 2015 -> 2020 wrote source rasters, **BII_change.tif**, and **zonal_stats.csv**.



Good failure: the original 2020 -> 2021 input now gets past the dependency blocker, then correctly fails because valid BII years are 2000, 2005, 2010, 2015, and 2020.

Status in one sentence

The dependency blocker is fixed and verified far enough for Team Platypus to treat BII browser execution as a real workflow again - while still separating runtime success from ecological interpretation.

Why this matters for Team Platypus

This repair is a useful teaching case because it shows the difference between a fluent demo and a governed workflow. A good assistant cannot hide infrastructure failures, invalid years, missing evidence, or source boundaries behind confident language.

1. Reproducibility

Before the fix, the pipeline never reached the biodiversity analysis. After the fix, the run reaches STAC/raster processing and produces inspectable artifacts.

2. Platform fit

Exact conda pins can be brittle on ARM. For this stack, the more robust pattern is a validated ARM rbase route plus source installs in the runner image.

3. Correct refusal

The 2021 rejection is not a bug. It proves the route now gets far enough for domain validation to work: valid BII years remain 2000, 2005, 2010, 2015, 2020.

4. Evidence handoff

Team Platypus should show the chain: input -> scoped route -> generated files -> uncertainty -> expert review. That is stronger than simply saying 'it runs'.

Suggested Team Platypus checklist

RUN	Use valid BII years only: 2000, 2005, 2010, 2015, 2020.
TRACE	Record the exact browser input, script route, runner image, and output paths.
CITE	Show generated artifacts: source rasters, BII_change.tif , zonal_stats.csv , and output.json .
BOUND	Do not claim ecological validity from runtime success alone. Keep expert review in the loop.
REFUSE	When a year, dataset, source, or local conclusion is unsupported, narrow or refuse instead of smoothing it over.



The lesson: a government-grade biodiversity assistant is not just a chatbot. It is a workflow that preserves provenance, source limits, uncertainty, and human judgement from the first dependency solve to the final answer.

Source notes for facilitators

Prepared from Forge's root-cause and verification note supplied by Hans. Framed for Team Platypus against the challenge standard: bounded evidence, visible uncertainty, source boundaries, and human review before real-world use.

This explainer does not claim endorsement, commissioning, production readiness, or ecological validity of outputs beyond the stated smoke tests.